

基于 Jittor 的时序图候选排序： 赛道一 A 榜结果复现技术报告

宋文韬

sader 团队 • 赛道一 • A 榜第 7 名

摘要：本报告给出赛道一 A 榜记录结果的完整代码说明、模型训练协议、锁定推理过程、环境配置和审计边界。任务由两个时序图候选排序数据集组成，每个查询均在官方给定的 100 个候选内预测相对概率。Dataset1 的原始成员为两个 Jittor 时序图排序网络；Dataset2 的原始成员覆盖 Jittor MultVAE、RecVAE、BM25-BPR、候选池排序器、集合排序器、多切片 Transformer 和 warm residual。A 榜记录结果采用一条可验证的锁定重建路径：读取官方 `data_A.zip`、包内冻结分数工件、Jittor BPR32 检查点与固定后处理规则，生成规范 `result.zip`。构建器对官方数据、冻结工件、模型检查点、规则文件和最终 ZIP 均实施 SHA-256 约束；任一环节发生差异即失败退出。所有深度学习计算严格使用 Jittor，不使用外部数据集，不读取测试标签。报告以当前 `code/main.py` 及保留训练源码为准，明确区分“记录结果的精确重建”和“从官方数据重新训练得到的新模型”。

关键词：时序图；候选排序；Jittor；图推荐；BPR；可复现性；提交审计

图 1 记录结果的可审计重建流程

01 • 官方输入 data_A.zip 只读取训练记录与测试候选、来源、时间字段。	02 • Jittor 基座 冻结分数与 BPR32 保留的基座结果与 Jittor 用户向量检查点均受哈希约束。	03 • 锁定规则 候选内结构残差 来源支持、精确组支持和跨来源社区信号在候选内计算。	04 • 规范输出 result.zip 固定序列化、CRC 与 SHA-256 验证，失败即停止。
--	---	---	--

1. 团队信息

表 1 参赛与联系信息

字段	内容
团队名称	sader
赛道与 A 榜排名	赛道一；第 7 名
A 榜最优总分	1.521072794155721
联系人	宋文韬
联系方式	微信 laoshan_song；电话 18984157192

交付包根目录为 `contest1_sader_007/`。根目录中包含 `code/`、`requirements.txt`、本说明文档、可执行入口、冻结模型工件和 `MANIFEST.sha256`。包不包含官方数据、外部数据、测试标签和最终提交结果；这些边界由 `code/tools/audit_exact_package.py` 在运行时检查。

报告采用时序图学习论文常用的“问题定义—核心模块—训练协议—复现验证—讨论”组织方式。版式层级参考 Yi 等人的 `Future Link Prediction Without Memory or Aggregation` (arXiv:2505.19408)：采用明确的标题区、摘要、方法流程、连续正文与图表编号；本文不复用该论文的正文、模型定义、图像或实验结论，算法、参数和所有结论均来自本项目现有代码与工件。

2. 项目概述

2.1 任务定义与数据边界

赛道一要求对时序交互图中的候选节点进行逐行排序。对于第 i 条查询，官方测试数据提供来源节点 s_i 、时间 t_i (Dataset2 中显式使用) 以及候选集合 $C_i=\{c_{i1},\dots,c_{i100}\}$ 。模型输出长度为 100 的非负向量 p_i ，并满足每行概率和为 1。提交 ZIP 必须含 `dataset1.csv` 和 `dataset2.csv` 两个成员；锁定构建器分别要求它们具有 $61,051 \times 100$ 与 $153,420 \times 100$ 的形状。

时序约束是本任务的关键。原始训练脚本先基于官方训练交互构造历史结构、时间衰减和来源—候选关系；推理只访问官方测试表中的候选、来源和必要时间列。候选表允许用于候选集合的共现与候选内归一化，但不允许引入候选域以外的外部节点信息，也不允许读取或推断隐藏标签。本项目所有静态审计均围绕这一数据边界进行：包内不能出现 `data_A.zip` 或最终 `result.zip`，构建器必须验证传入官方档案的已知哈希。

从概率建模角度看，原始成员为每个候选给出未归一化分数；后续融合、候选内 `qnorm` 与 `softmax` 的作用是将不同成员的数值尺度对齐，而不是新增候选或改变训练标签。设 z_{ij} 表示查询 i 的候选 j 的融合 `logit`，则提交概率由候选内 `softmax` 得到。这样既保留排序单调性，也使每一行满足比赛格式要求。

$$p_{ij} = \frac{\exp(z_{i0ij})}{\sum_{k=1} \exp(z_{ik})}$$

2.2 双数据集建模思路

Dataset1 更强调来源节点在时间图中的局部历史、近期邻域和候选支持关系。原始训练实现把来源、候选、时间、频次、近期交互和图传播特征组合为固定宽度输入，再用 `Jitter embedding` 和多层感知机学习候选分数。为了降低单个随机初始化的方差，训练协调器使用两个固定种子生成图排序成员，并在打分阶段集成。锁定重建阶段不重新训练这两个成员，而是在保留的 Dataset1 分数空间上应用来源支持后处理；这使记录结果的规则与输入工件都可以被逐项校验。

Dataset2 的交互规模与时间结构更复杂。项目把稀疏历史表示、成对排序、候选集合上下文与多时间切片结构分别建模：`VAE` 系列提供全局稀疏偏好表示；`BM25-BPR` 使用时间衰减和负采样学习排序偏好；`pool`、`set` 和 `Transformer` 成员从官方候选池中抽取不同粒度的上下文；`warm residual` 用于补充

热节点相关的候选内残差信号。锁定路径在冻结基座上保留精确候选组支持与社区余弦残差，避免重新训练时随机性、编译器差异或历史调参状态影响记录结果的可验证性。

2.3 模型族与职责划分

表 2 原始训练模型族及其功能定位

成员	建模对象	Jittor 实现位置
Dataset1 图排序成员	时序图统计、来源—候选 embedding、近期交互与候选排序	raw_training/dataset1/
MultVAE / RecVAE	稀疏历史交互的隐变量重构与偏好表示	dataset2/train_vae_jittor.py
BM25-BPR	时间衰减、BM25 加权和正负候选排序	dataset2/train_bpr_jittor.py
pool ranker	候选池及切片上下文的排序信号	dataset2/d2_pool_ranker*_jittor.py
set / multislice set	候选集合的置换不变表示和多时间切片融合	dataset2/d2_*set*_jittor.py
multislice Transformer	候选集合内的交互注意力与跨切片上下文	dataset2/d2_multislice_transformer*_jittor.py
warm residual	热节点候选的附加候选内残差	dataset2/d2_warm_residual*_jittor.py
community BPR32	跨来源同候选用户对 Jittor 余弦社区信号	train_community_bpr_jittor.py

模型族的设计不是把所有成员机械相加。VAE 和 BPR 在历史交互表达上互补：前者面向稀疏向量的整体重构，后者直接优化排序相对关系；pool、set 与 Transformer 在相同候选池上形成不同的信息聚合归纳偏置；社区 BPR32 则只在锁定后处理的特定跨来源共现条件下启用。通过将这些角色拆分，训练代码能够单独记录每个模型文件、参数和哈希，推理代码也能明确每个中间分数的来源。

2.4 Dataset1：时序图排序与来源支持

Dataset1 的核心实现位于 `code/raw_training/dataset1/run.py`。该文件构造时间排序的历史交互，维护来源频次、候选频次、最近出边、最近入边、来源—候选对次数和最近时间等统计量。为了让候选排序能同时使用局部图结构和来源个性化偏好，网络包含来源 embedding、目标 embedding、来源/目标偏置以及多层 Jittor 全连接层；可选历史分支将近期候选 embedding 聚合为短期兴趣向量。训练时以每条查询中的候选集合为单位计算交叉熵型排序目标，避免把不同查询的候选概率混在同一归一化空间。

协调器 `raw_training/main.py` 为两个种子 20260705 和 20260715 调用 `train_d1_cf_jittor.py`。固定训练参数为 `groups=80000`、`valid=20000`、`epochs=16`、`batch=512`。这些数值分别约束训练查询数量、验证查询数量、完整数据迭代次数和每步候选批量；它们不是文档层的描述性数值，而是协调器实际传入训练脚本的命令行参数。打分阶段由 `score_d1_ensemble_jittor.py` 读取两个模型，对官方候选表写出融合分数。

记录结果的后处理实现为 `code/dataset1/dl_source_support_postprocess.py`。该模块先读取官方测试候选和冻结结果中的 Dataset1 概率，验证 CSV 序列化闭环，再对来源支持、唯一胜者、候选最大支持与差距执行确定性计算。锁定规则参数存于 `experiments/dl_source_support_locked.json`，构建器把文件哈希记录在 `build_audit.json`。这种分离使图模型训练和规则层的责任边界清晰：训练脚本负责从历史交互学得基座分数，后处理脚本只在既定候选上作审计友好的单调整。

2.5 Dataset2：多成员基座与社区残差

Dataset2 的原始训练从官方训练表中读取 `src`、`dst` 与 `time`。VAE 脚本按生产历史边界构造用户—目标稀疏交互矩阵，在 Jittor 中训练编码器/解码器；BPR 脚本从时间衰减后的交互中采样正负目标对，学习来源与候选的嵌入表示。生产 BM25-BPR 的固定设置为 factors 256、epochs 20、batch 32768、negatives 8、learning rate 0.002、regularization 0.0001，时间衰减窗口为 365 天。更高容量的 RecVAE 配置使用 hidden 800、latent 400 和 batch 96，其余 VAE 配置使用 hidden 600、latent 200、batch 128 与 dropout 0.5。

候选上下文成员并不访问额外社交网络或外部特征。pool ranker 在官方候选池和历史边上构造切片；set ranker 对 100 候选集合构造置换不变的聚合表示；multislice set 与 multislice Transformer 在多个时间切片上重复该过程并融合输出；warm residual 使用同一候选结构产生热节点相关的残差信号。原始训练的 `main.py` 会为这些成员生成独立的 `.npz` 文件，然后通过 `build_result.py`、`build_set_v2_result.py` 等脚本逐步合成为新的基座 ZIP。每个阶段均在调用方新建的模型或输出目录中执行，避免复用已有结果。

锁定重建从 `base_result.zip` 读取 Dataset2 的基座概率。`code/dataset2/d2_exact_group_postprocess.py` 首先构建同一 `(src,time)` 组内的跨行候选支持；随后 `build_community_residual_submission.py` 基于同一时间、同一候选的出现位置建立跨来源用户对。对于能够在 BPR32 检查点中匹配到的用户，脚本在 Jittor 中计算 32 维用户向量归一化和余弦值；相同来源对和未知用户对会被显式排除。余弦值上分位的用户对在两端候选单元产生社区存在标记，最终经过候选内 qnorm 加入分数。

$$Z_{ij} = \text{qnorm}(\log p_{ij}^{\text{base}}) + 0.05 \cdot \text{qnorm}(\mathbb{I}[s_{ij}^{\text{exact}} > 0]) \\ + 0.02 \cdot \text{qnorm}(\mathbb{I}[c_{ij}^{\text{community}} = 1])$$

该表达式中的两个系数、候选宽度、时间稳定性要求、批量大小和检查点哈希都被代码固定。社区项不是利用标签的相似度学习，也不是把测试候选视为训练监督；它仅对官方候选结构中的跨来源重复出现关系进行无标签标记，并以冻结的 Jittor BPR 用户表示决定哪些标记属于高相似度社区。

2.6 方法创新点与设计取舍

第一，项目将“训练期的图表示学习”和“提交期的候选内结构校正”分开。训练期模型承担从历史交互学习偏好的职责；提交期规则只允许读取官方候选、时间和冻结分数，并通过哈希绑定输入。相比将所有启发式混入训练网络，这种设计更容易说明数据边界，也使最终提交能被独立重建。第二，Dataset2

社区残差不采用全局热度加分，而是以时间和候选为键建立局部共现，再排除同来源与未知用户，从而把热节点信息约束在可解释的候选对上。

第三，所有融合都在单个查询的 100 个候选内部完成。候选内 qnorm 使不同成员的绝对分值不会直接互相压制，softmax 则保证输出满足比赛格式。第四，构建器从输入档案哈希、检查点字段形状、概率有限性、CSV 内容、ZIP CRC 到最终 SHA-256 设置多层失败条件。该策略的目标不是在异常环境下继续生成近似文件，而是在任一无法证明一致性的环节停止，以避免把非记录结果误报为记录结果。

3. 代码结构说明

3.1 目录分层与责任边界

提交包采用“锁定重建层—原始训练层—工具与审计层”的分层结构。锁定重建层保存对 A 榜记录结果必要的最小工件与确定性代码；原始训练层保留从官方训练数据构建各 Jittor 成员的完整实现；工具层负责环境预检、静态框架审计、打包和报告生成。这样的目录布局避免把训练期生成的大量模型塞进提交 ZIP，同时保留训练脚本、推理脚本和核心算法逻辑供人工审核。

```
contest1_sader_007/
├─ code/main.py # 统一命令入口
├─ code/build_community_residual_submission.py # 锁定结果构建器
├─ code/dataset1/ # Dataset1 后处理
├─ code/dataset2/ # Dataset2 精确组后处理
├─ code/raw_training/ # Jittor 原始训练与新推理
│   └─ dataset1/ # 图排序训练与集成打分
│   └─ dataset2/ # VAE/BPR/pool/set/Transformer
│   └─ legacy_dataset2/ # 保留的 Jittor 组件
│   └─ main.py # 训练/推理协调器
├─ code/tools/ # 预检、审计、打包和报告工具
├─ models/model_bpr32_prod_community_jittor.npz
├─ base_result.zip
├─ experiments/dl_source_support_locked.json
└─ run_verify.sh / run_inference.sh / run_train.sh / run_fresh_inference.sh / run_raw.sh
```

`code/main.py` 是面向复现者的统一入口。它不自行实现模型，而是把参数以原样方式传给根目录 shell 启动脚本，使 CUDA 缓存、非覆盖约束、环境预检和历史命令契约保持一致。其 `verify` 子命令对应记录结果的精确重建；`infer` 跳过预先的静态包审计但保留锁定构建；`train` 训练原始 Jittor 模型；`fresh-infer` 对新模型推理；`raw` 组合训练、推理和收据验证。

3.2 锁定重建调用链

```
python code/main.py verify --data DATA --output OUT
-> run_verify.sh
-> sha256sum -c MANIFEST.sha256
-> audit_exact_package.py --root . --data DATA
-> run_inference.sh
-> prepare_cuda_runtime.sh
-> check_exact_environment.py
-> build_community_residual_submission.py
-> d1_source_support_postprocess.py
-> d2_exact_group_postprocess.py
-> Jittor BPR32 cosine residual
-> OUT/result.zip and OUT/build_audit.json
```

该链路先检查包自身，再检查数据，再检查 CUDA/Jittor 环境，最后执行计算。`run_inference.sh` 要求输出目录不存在；它把 `ML_CACHE_ROOT`、`HOME` 和 `JITTOR_HOME` 定向到调用方指定的可写缓存根，随后调用 CUDA/cuDNN 预备脚本。构建器读取基座和检查点后再次验证哈希，只有最终文件的 SHA-256 等于记录值时才打印成功信息。由于 ZIP 写出采用固定成员名与固定顺序，字节哈希是对结果内容和封装方式的联合验证。

3.3 数据处理模块的逻辑

Dataset1 后处理模块从官方测试表提取来源和 100 个候选。它使用稳定排序检查候选唯一性，加载冻结基座中的概率矩阵，并重新序列化以确保数值文本没有被非规范格式改变。来源支持函数聚合同一来源的候选出现情况，输出每个候选的支持量、唯一胜者和最大支持差；锁定配置决定何时在 logit 空间施加提升。所有数组形状和输出哈希在构建器中再次核对。

Dataset2 后处理模块从官方测试表读取 `src`、`time` 和候选矩阵。精确组支持函数使用 `(src,time)` 键把相关行聚类，在不同记录间计算同候选是否出现。`qnorm` 将一个 100 维候选向量中心化并按行尺度归一；`softmax` 只在同一行产生概率。社区函数使用候选单元索引保存重复出现位置，而不是将整个测试图转化为无约束的全局图，从实现层面限制了测试结构的作用范围。

3.4 原始训练和新推理协调器

`code/raw_training/main.py` 是原始训练协议的唯一协调器。训练 Dataset1 时，它依次调用两个种子配置；训练 Dataset2 时，依次调用生产 VAE、BPR、不同历史切分、pool、set、多切片 set、多切片 Transformer、warm residual、legacy 重建和社区 BPR。每个子过程通过 `subprocess.run(..., check=True)` 启动，任何失败都使协调器退出。训练结束后，协调器对模型目录建立递归 SHA-256 清单，并写入 `training_manifest.json`；新推理则写入 `inference_manifest.json`，其中绑定数据、源码、需求文件和模型工件。

3.5 Dataset1 原始图排序网络的输入、计算与训练目标

Dataset1 的原始实现并非把边列表直接交给不可解释的黑盒，而是先由 `dataset1/run.py` 将历史边转化为可复核的时序图统计，再由 Jittor 网络学习统计特征与节点身份之间的非线性组合。读取阶段只打开官方压缩包中的 `dataset1/train.csv` 与 `dataset1/test.csv`。训练边按时间稳定排序；若训练表包含

官方 `split` 字段，代码将 `split=0` 作为历史段、`split=1` 作为目标段；若缺少该字段，则将有序边按比例切出历史与目标。后续所有历史统计都只能由历史段写入，目标段只用于构造监督查询和验证查询。这个先历史、后目标的顺序是避免时序倒灌的第一层约束。

统计对象包含来源节点累计次数、目标节点累计次数、来源和目标的出现时间、来源—目标对的出现次数、来源—目标对的首次和最近时间、每个来源最近出边序列、每个目标最近入边序列，以及按目标节点累计的小时和星期分布。最近序列使用固定长度的双端队列，当前实现最大保存 64 个历史邻居；输入到短期兴趣分支时再截取其中长度为 8 的最近目标。对每一条“来源、候选、查询时刻”三元组，代码根据这些缓存生成频次、对频次、相对新近度、局部邻居重叠、周期性匹配及候选池频次等特征。候选池频次只来源于官方测试表所给的 100 候选列，并且在每条查询的候选内部参与排序，不产生任何未提供候选。

新近度以距离上次事件的真实时间差为输入。对应实现将天级差距压缩为 $1/(1+\log_{1p}(\text{delta}/86400))$ ，以便近期交互的变化更敏感而很早历史不会造成无界数值。该压缩函数不预测标签，也不假定某一个具体候选必然正确；它只是把图边的时间戳映射成单调递减数值。对频次、来源度和目标度同样采用对数或归一化处理，原因是原始交互次数具有长尾分布，直接输入会使极少数高频节点主导梯度。

基础网络 `Net` 使用来源 embedding、目标 embedding、来源偏置、目标偏置和一个三层 Jittor MLP。MLP 维度为“输入特征维度→128→64→1”，中间使用 ReLU，首层之后使用 0.05 dropout。身份交互项为来源向量和目标向量的内积，偏置项补偿不同节点的总体出现倾向。这样，手工时序图特征可以描述“当前查询的局部状态”，embedding 内积可以表示“这个来源与这个候选的稳定兼容性”，MLP 则负责学习两类信号间的非线性权衡。所有参数均通过 `jittor.nn.Embedding`、`jittor.nn.Linear` 和 Jittor 优化器建立，不存在 PyTorch 或其他深度学习框架的并行实现。

带历史的 `NetAttn` 在同一主干之上引入候选条件的短期兴趣。它把当前候选 embedding 线性投影为 query，把来源近期目标的 embedding 投影为 key/value；注意力分数由 query 与 key 的缩放内积再减去学习到的真实时间间隔项得到。无效历史位置先用大负值遮蔽，softmax 后再乘以有效掩码，因此空历史不会被伪造为某个节点的偏好。加权 value 上下文与当前候选向量的内积被加入最终分数，历史 embedding dropout 为 0.2，注意力 dropout 为 0.3。相比只对历史节点做平均，这一分支允许同一个来源在不同候选上关注不同的近期事件，也明确将时间间隔而非序号位置写入打分。

训练样本以“一个正目标加 99 个候选池负例”的 100 候选组构成。`sample_groups` 从目标边中选择正例，将正目标写入随机列，再从官方候选池采样其余候选；若随机候选与正目标重复，代码会重新采样以保持正例索引清楚。网络在同一组的 100 个候选分数上计算交叉熵，而不是将多个查询的候选混合做一个全局 softmax。因而训练目标与评测时的相对排序接口一致：模型的责任是把给定来源和时间下的真实目标排在同组 100 个候选之前。

验证阶段沿时间从训练目标之后切出，避免随机打散导致未来事件进入历史。所有随机性均由 Python `random`、NumPy 生成器和 `jt.set_global_seed` 同时固定。协调器使用 20260705 与 20260715 两个种子训练两个成员并在打分阶段集成；两个成员之间的差异来自初始化和候选采样路径，而不是数据源、标签边界或深度学习框架差异。这样的双种子设计降低了单个初始化对候选排序的偶然影响，同时保持模型规模与训练成本在可控范围内。

3.6 Dataset2 稀疏交互表示：时间衰减、VAE 与 BPR

Dataset2 的训练数据读取逻辑集中在 `train_vae_jittor.py` 和 `train_bpr_jittor.py`。两个脚本都先从官方压缩包读取 `dataset2/train.csv` 的 `src`、`dst`、`time` 三列，再用固定的历史端点截断数据。生产切片的端点是 1296345600；另保留 y2009 和 strict 等较早端点成员。切片不是从测试数据反推得到的时间点，而是训练协议中明确传入的常量。对每个切片，历史中出现的来源和目标被稳定映射为紧凑索引，原始业务 ID 仍以数组形式保存在模型检查点，以供后续候选打分时进行二分映射。

VAE 路径先建立来源—目标的稀疏行表示。对于历史边 e ，其权重为指数时间衰减：距离当前历史段末端越远，权重越小；衰减尺度由 `decay-days=365` 固定。实现没有把整张稀疏矩阵交给第三方深度学习库，而是以 `indptr`、`indices`、`values` 三个 NumPy 数组保存 CSR 风格行结构，并在每个训练 batch 将选中的行展开为 float32 稠密矩阵后送入 Jittor。这一做法使稀疏数据的存储逻辑和神经网络计算图的责任边界清晰：前者只处理索引与时间权重，后者承担可导的编码、采样、重构和优化。

MultVAE 的编码器先对每个来源行作 L2 归一化，再以 0.5 dropout 扰动输入，经过 `hidden=600` 的 `tanh` 层得到均值和对数方差。采样使用重参数化形式 $z = \mu + \epsilon \cdot \exp(0.5 \cdot \log \text{var})$ ，解码器经 `latent=200` 到 `hidden` 再到全目标空间的线性输出给出候选 `logit`。训练损失由加权的多项重构项和 KL 项组成，KL 权重从 0 线性退火到 0.2，退火步数为 2000。该退火避免训练初期 KL 项过强导致隐变量被忽略；梯度范数裁剪为 5.0，优化器为 Jittor AdamW，学习率 0.001、权重衰减 $1e-5$ 。

RecVAE 采用更深的残差编码器。它由五层 `hidden` 维线性层和 `LayerNorm` 组成，激活函数为 `swish`；每层在输入上累积前面隐层的残差。与只使用标准正态先验的 MultVAE 不同，RecVAE 同时保留一个停止梯度的先验编码器，并在 KL 中混合标准正态、历史先验和高方差分量。实际实现中三项混合系数为 0.15、0.75 和 0.10，KL 系数 `gamma` 为 0.005。每个 cycle 先训练编码器若干阶段，再固定编码器训练解码器，并用当前编码器更新先验；这一交替过程旨在稳定更强容量下的后验学习。生产配置同时保留 `hidden/latent=600/200` 与 `800/400` 两种规模，前者为主力稀疏表示，后者提供不同容量的互补视角。

VAE 检查点不是只保存一个不可读的序列化对象。导出程序将用户 ID、目标 ID、导出的用户表示、解码器目标向量、目标偏置、模型类型、切片元数据以及所有 Jittor state dict 参数写入压缩 NPZ。这样，独立打分器既可以通过“用户表示与候选目标向量内积加偏置”的紧凑接口计算候选分数，也可以让审核者检查完整的 Jittor 参数是否存在。写入前模型切换到 `eval` 模式，推理编码过程关闭输入扰动，并按 batch 分批导出，避免训练时的随机采样进入候选分数。

BM25-BPR 路径直接优化来源对正负候选的相对偏好。历史交互先按距切片端点的指数衰减得到原始置信度，再按来源行长度进行 BM25 归一化。当前固定参数为 `k1=100`、`b=0.8`；目标的逆文档频率由出现该目标的来源数量计算。归一化后置信度被裁剪到 `[0.05,20.0]`，既避免极罕见目标造成数值放大，也不让高频来源的长历史完全淹没较新的正边。它是历史交互的权重变换，不读取或估计测试标签。

Jittor BPR 模型为来源和目标分别配置 `factors=256` 的 embedding。对一个 batch 内的每个正对，代码随机采样 8 个负目标，计算正分数与负分数的差，并最小化 `softplus(-(positive-negative))` 的置信度加权均值；L2 正则项系数为 $1e-4$ 。每轮随机置换正对，`batch=32768`，`epochs=20`，学习率为 0.002。梯度经过 Jittor AdamW 和范数裁剪后更新。与点式分类不同，这个损失只要求正目标相对于随机负目标更高，因此适合候选排序而不要求把每个目标建模成独立二元分类任务。

生产 VAE 与 BM25-BPR 都严格设置 `jt.flags.use_cuda=1`，并在 Jittor 没有 CUDA 能力时抛出错误。此限制在训练阶段的重要性不只是速度：它避免一个成员在 CPU 路径、另一个成员在 GPU 路径上生成难以解释的运行差异。每个模型文件包含训练切片、衰减天数、维度、随机种子、epoch 数等元数据；协调器再把文件哈希纳入训练清单。因此，当重新训练生成的模型与记录工件不同，清单会将两者区别开来，而不会覆盖或伪装为记录工件。

3.7 候选集合排序成员与多时间切片实现

VAE 与 BPR 均以“来源历史→目标分数”为主，而赛题的最终输入还包含每行固定的 100 候选集合。项目因此保留 pool、set、多切片 set、多切片 Transformer 与 warm residual 这一组候选集合成员。它们的共同原则是：只在官方给定候选列中产生位置分数，并把监督信号组织成同样大小的候选组。它们并不扩展候选域、也不调用外部知识图谱或社交关系库。候选集合成员的价值在于学习“在这组互相竞争的候选中，哪些局部统计组合更有辨别力”，而不是用额外数据直接猜测答案。

`d2_pool_ranker_production_jittor.py` 为三个固定种子训练轻量 pool ranker。代码先从 VAE/BPR 等基模型中生成训练与官方测试的候选特征，再将每条查询的特征组织为 `[batch,100,feature_dim]` 张量。网络对每个候选应用共享 Jittor MLP，训练时用 100 类交叉熵；共享参数使模型对候选列的绝对位置不敏感。生产脚本的默认 `batch=512`，候选采样种子为 20260717；检查点保存每个 seed 的参数和元信息。三个独立种子的输出在后续阶段作为一个小型集成，而不是依赖单一模型的局部极值。

set ranker 的实现使用候选间的集合上下文。对于单个候选，它不仅接收自身的基础分数和结构特征，还与同一 100 候选集合的聚合表示拼接。由于聚合使用候选维度上的无序操作，模型强调集合成员关系，而非把第 1 列、第 2 列等位置当成语义特征。`d2_v2_set_production_jittor.py` 以 6 个检查点为默认输入，按 64 和 96 的隐藏维度、各 3 个种子形成不同容量的集合成员；训练优化器同为 Jittor AdamW，学习率 0.001、权重衰减 $1e-5$ ，组内交叉熵保证输出的监督接口与评测接口一致。

多切片 set 对同一套候选特征加入历史端点差异。生产切片偏重相对较新的交互，y2009 和 strict 切片提供更早的历史视角；对每个候选，模型可以比较“在近期历史中有多强”与“在更长历史中是否稳定存在”。多切片代码先要求各基础模型文件存在，再逐块读取特征并训练/保存 set 网络；缓存目录与输出目录均由调用者显式传入。不同切片模型不是通过访问未来边得到差异，而是从官方训练边在不同历史截止点分别训练得到。

多切片 Transformer 成员位于 `d2_multislice_transformer_raw_jittor.py`。其 `TransformerBlock` 和 `TransformerSetRanker` 由 Jittor 线性层、注意力相关张量操作、残差与归一化模块构成。输入仍为每行 100 候选的结构化特征，不使用任何文本预训练模型或非 Jittor attention 实现。Transformer 的作用是让一个候选的分数参考同组其他候选在多个时间切片下的特征，而集合网络的作用是提供更轻量的无序聚合。两类归纳偏置在集成中并存，降低单一集合机制在某些来源群体上失效的风险。

warm residual 模块专门面向热节点的候选内残差信号。脚本首先依据已有切片成员与官方候选构造特征和训练标签，再训练一个 Jittor MLP 输出 100 个候选的分数。训练中使用逐行交叉熵，并根据 warm 掩码只对适用行计入损失；掩码分母加 $1e-6$ 以防没有热样本的 batch 产生除零。该模块不把“热”解释为答案，而是把它作为一个需要与基座分数共同判断的条件，因此最终只以残差的形式融入候选内排序。

上述成员均以 NPZ 存放 Jittor 参数和必要的训练元数据，最终基座由 `build_result.py`、`build_set_v2_result.py`、`build_final_submission.py` 等脚本按既定顺序组合。原始训练路径会将每一步的输入模型路径、输出文件、文件哈希和执行命令写入清单。工程上将“训练一个成员”“计算一个成员分数”“把分数融合到基座”拆成单独脚本，是为了使失败定位和审核可追溯：审核者可在某个阶段检查数据边界、模型参数和输出形状，而无需从一个过长的总脚本中反向猜测依赖关系。

3.8 锁定重建的 Dataset1 结构规则

记录结果的 Dataset1 最终层由 `d1_source_support_postprocess.py` 执行。该文件首先对官方测试 CSV 的 header、行数、每行 100 候选宽度、ID 范围、候选行内唯一性和候选全集大小进行校验。测试文件内容必须与代码中固定的 SHA-256 相同，冻结基座 ZIP 必须只包含 `dataset1.csv` 与 `dataset2.csv`，并通过 CRC、成员顺序和成员内容哈希检查。这样做的目的不是把数据藏在代码中，而是防止同名但内容不同的档案被误作为官方输入。

来源支持定义为同一来源的不同测试查询中，某候选出现的行数。实现把每一个来源—候选单元编码为 64 位键 `src*2^32+candidate`，对所有单元做唯一化计数，再还原为与测试矩阵相同的 [61051,100] 支持矩阵。对每一行，脚本选择支持最多的候选，并计算第一名与第二名支持的差距。只有最大支持不小于 4、且领先至少 2 行时，该候选才具备规则资格；这两个阈值和动作名称都在 `experiments/d1_source_support_locked.json` 中固定，运行时同时校验规则文件哈希。

资格成立并且支持胜者不是基座 top1 时，脚本转到 logit 空间提升该候选：先取基座概率的对数，再把胜者的 logit 设为当前行最大 logit 加 1.0，最后在该行 100 候选上重新 softmax。这样的操作保证胜者成为 top1，同时明确验证其余 99 个候选的相对排序不变。它不改变候选集合、不引入新节点，也不在跨来源间传播信息；规则只利用“该来源在官方候选表的多个查询中反复出现的候选”这一确定性结构。诊断函数还计算改变行数、top-k 重叠、行和误差和序列化前后的变化位置，帮助审核者确认规则影响的范围。

Dataset1 规则的完整性来自三个层次：训练期的双 Jittor 图成员构成冻结基座；结构规则通过固定 JSON 配置与固定测试哈希约束输入；最终 CSV 使用八位小数的确定性序列化。任何一个部分变化都会导致固定的 Dataset1 成员 SHA-256 不匹配，从而使构建器停止。因而“图模型学习的得分”和“候选表上的确定性排序修正”可以分别检查，也不会构建时静默替换为新的实验结果。

3.9 锁定重建的 Dataset2 精确组与社区残差

Dataset2 的第一个确定性残差由 `d2_exact_group_postprocess.py` 实现。查询组键定义为精确的 `(src,time)`，不是近似的时间桶，也不把不同来源合并。对于每个组内的候选，函数计算同一目标是否出现在至少一条其他行中；同一行内的重复位置先被稳定排序和去重，再计入不同“行”的支持，因此一个候选在当前行出现两次不能凭自身获得额外证据。实现用 `group*ITEM_KEY_BASE+candidate` 形成全局可排序键，利用 NumPy 的 unique 与 bincount 得到每个候选单元的其他行支持数，之后通过原排序索引恢复为原来的 100 列顺序。

精确组残差只使用二元条件 `support>0`，而不是按支持次数持续放大。基座概率先做对数截断和候选内 qnorm：每行减去该行均值，再除以标准差加 $1e-6$ ；二元支持向量也在同一行独立 qnorm。两者以固定权重 0.05 相加后做稳定 softmax。候选内归一化的含义是把“某个成员的绝对分数范围”转换为“该

成员在本查询 100 候选中的相对偏好”，避免包含更多活跃候选的行因分数尺度而被过度推高。脚本拒绝其他权重作为输入，并在诊断中记录活动行比例、支持单元比例、top1 变化率、top5 重叠和行概率和误差。

第二个残差为跨来源社区标记，且其中的向量运算严格由 Jittor 完成。构建器先读取哈希固定的 BPR32 检查点，要求其 `kind=bm25_bpr`、`lineage=d2_cross_src_community_prod_bpr32_v1`、`history_end=1296345600`、`factors=32`，并核查用户/目标 ID 的排序、参数数组形状、float32 类型、有限性以及 `state dict` 参数的逐元素一致性。检查点只提供历史训练学得的用户表示；在最终社区计算中，只会取到与官方测试来源 ID 匹配的向量，未知用户被明确排除。

社区对的构造以“同一测试时刻、同一候选目标”为键，即 `time_group*ITEM_KEY_BASE+candidate`。每个候选在一行内先去重，只有来自不同来源的同键候选单元才可能配对；同来源对被剔除，任一来源不存在于 BPR32 模型中的对也被剔除。官方测试时间要求非递减，以保证时间 `group` 的构造稳定。对于剩余对，代码将用户 `embedding` 分批送入 Jittor，计算带 $1e-12$ 数值保护的 L2 归一化和点积余弦；批大小固定为 131071。整个路径由 `jt.array`、逐元素乘法、`sum` 和 `jt.sqrt` 完成，结果回到 NumPy 只用于索引、阈值和序列化。

社区余弦不是对所有候选加相似度分数。构建器在已知的跨来源对上取余弦的最高 10% 分位阈值，将达到阈值的两端候选单元标为存在；若某行内部存在相同候选，其标记会通过稳定排序传播到该行的所有重复位置。随后该布尔存在矩阵按行 `qnorm`，并以固定系数 0.02 加到“基座 + 精确组残差”的 `logit` 上。这样，社区项的行为可解释为一个很小的、仅在强同刻同候选关系出现时触发的候选内方向信号，而不是对每个用户或每个热门目标施加全局偏置。

最终的锁定公式为：`qnorm(log(base))`
`+0.05*qnorm(exact_other_row_support>0)+0.02*qnorm(cross_source_community_top10pct)`。三个项都在每行的 100 候选内规范化，且所有权重作为 Python 常量写入构建器。最终 `softmax` 的输入不包含测试标签、外部向量、网络请求或训练时临时调参文件。构建器还要求重算的 `Dataset2 CSV` 流式 SHA-256 与固定值一致，并要求最终 ZIP 的字节 SHA-256 与记录结果值一致。若 `qnorm`、候选顺序、Jittor 余弦、浮点序列化或 ZIP 元数据任何一项发生差异，构建不会将近似输出误记为通过。

3.10 融合、数值规范化与输出可判定性

候选排序系统通常容易出现一个工程问题：不同成员的原始 `logit` 范围不相同，直接相加等价于让数值尺度最大的成员主导所有查询。项目的锁定后处理因此采用候选内 `qnorm`。对一行 100 维向量 `x`，`qnorm` 的标准化定义如下；平均值和标准差都只在同一查询内部计算。该处理保持一行内的相对偏差结构，同时消除跨行、跨成员的平移和缩放。它不改变候选集合，也不把某一行的信息泄漏到另一行的尺度估计。

$$\text{qnorm}(x) = \frac{x - \mu(x)}{\sigma(x) + 10^{-6}}$$

`softmax` 实现先减去该行最大值再指数化，这一稳定化技巧防止概率极小或极大的行溢出。输入概率在取对数前被裁剪到不小于 $1e-12$ ；这只处理数值下界，不把 0 概率解释为标签。输出检查包含有限性、

非负性、行和、CSV 形状和 ZIP CRC。Dataset1 的序列化使用固定八位小数，Dataset2 的锁定写出同样逐行使用固定格式；ZIP 成员名固定为 `dataset1.csv`、`dataset2.csv`，时间戳固定为 1980-01-01 00:00:00，成员顺序和权限字段也明确设置。固定封装元数据是为了让“内容相同但 ZIP 二进制不同”不会被误当成精确重建。

从评审角度，行概率和正确只能说明输出格式正确，不能说明等于记录结果；反之，最终 ZIP 哈希匹配则同时证明 CSV 内容和固定封装都匹配。因此代码把这两类判断分开：新训练推理需要满足格式、有限性和收据清单；锁定重建额外要求每个中间成员哈希和最终 ZIP 哈希均精确相同。两种路径都保留，是为了既提供从官方训练数据重新训练全部 Jittor 成员的代码，也提供对 A 榜固定记录结果可独立审计的最小重建路径。

3.11 时间边界、样本构造与防止监督泄漏的实现约束

时序图任务中的关键风险不是普通的数组越界，而是模型在训练或特征生成时错误使用了本应位于未来的边。提交代码在多个层面显式处理该风险。Dataset1 的 `split_edges` 先对边按时间稳定排序，再从训练表的官方 `split` 或固定比例形成历史段和目标段；图统计对象只在历史段调用 `Stats.build`。来源度、目标度、来源—目标对频次、最近邻居和最近时间均由这一历史段累积，之后的正例查询不会回写到已经用于自身打分的历史缓存。验证查询又从目标段的末尾切出，因此验证时间不早于训练目标时间。

Dataset1 的候选训练组不把官方测试表中的任意列当作标签。测试候选池仅被用于提供困难负例的候选 ID 分布：每个训练正边仍来自官方训练表的目标列，正例位置由本地伪随机数生成器随机选择，交叉熵标签就是这个位置。这样构造的原因是比赛的推理接口固定为 100 候选排序，训练时必须学习“真实边相对于候选池内竞争者”的排序关系。代码对重复采样到正目标的负列做清理，避免一个训练组中出现多个等价正例而使标签定义含糊。

Dataset2 的切片边界由 `ORIGINS` 字典给出，生产端点、y2009 端点和 strict 端点均为源码常量。`load_matrix` 与 `load_pairs` 都使用 `train.time < origin` 筛选边，随后才建立 ID 映射、稀疏矩阵或 BPR 正对。因而一个较晚时间段的边不会被错误放入较早切片的 VAE/BPR 表示中。多切片成员只是并列使用这些不同历史长度所得到的特征，并不通过同一条边的未来标签为早期切片赋权。训练脚本读取的列被显式限制为 `src`、`dst`、`time`；它没有读取任何隐藏测试真值文件，也没有需要标签才存在的额外字段。

锁定重建则不重新建立训练监督。它读取官方测试候选矩阵所允许的 `src`、`time` 和候选列，并将其与包内冻结的历史模型工件相结合。Dataset1 来源支持只计数同一来源下候选在不同官方查询中的出现；Dataset2 精确组支持只计数同一 (`src,time`) 下候选在其他官方查询中的出现；社区残差只计算同一时间、同一候选在不同来源之间的历史用户表示余弦。这些操作都没有访问测试标签，也没有依赖某个候选的真实或错误状态。为了让这一点可复查，构建报告在成功时写入 `official_labels_opened=false`，静态审计还要求提交包中不存在官方数据档案和最终 `result.zip`。

测试候选的结构性使用需要准确区分“候选内无标签结构”与“把候选当监督”。本项目的所有结构项都满足前者：候选 ID 是否在同来源、同时间或同一时刻的另一行出现，是由测试输入本身确定的；该条件没有告诉程序哪个候选是真实目标。残差只在基座已有的概率排序上以 0.05 或 0.02 的固定量调整相对 logit，并通过候选内 softmax 返回比赛所需的概率。规则系数没有在构建期间接受外部传参，代码也没有网格搜索、读取 A 榜回传、读取隐藏标签或按行自适应选择系数的分支。

每个可能触及时间边界的实现点都保留了可读的输入路径。Dataset1 的训练入口是 `train_dl_cf_jittor.py`，图特征实现是 `dataset1/run.py`；Dataset2 的 VAE/BPR 输入位于 `load_matrix` 与 `load_pairs`；候选结构处理分别位于两个 `*_postprocess.py`；社区训练的输入与检查点生成位于 `train_community_bpr_jittor.py`。人工审核可从这些模块顺序读取，而不需要相信文档中的口头描述。报告中提到的训练协议、列选择、阈值和权重均可在对应源文件内定位。

3.12 工件谱系、命令契约与失败关闭策略

可复现提交不应依赖工作目录中“恰好存在”的临时文件，因此本项目将工件按作用分为官方输入、训练输出、冻结记录工件、规则和收据五类。官方输入是调用方传入的 `data_A.zip`；训练输出是 `train` 写入的新模型目录；冻结记录工件是包内 `base_result.zip` 和 BPR32 NPZ；规则是 Dataset1 JSON 配置与构建器中的固定常量；收据则是 `MANIFEST.sha256`、`training_manifest.json`、`inference_manifest.json` 或 `build_audit.json`。这些对象的角色不可互换：例如，基座 ZIP 可以作为锁定重建的输入，但不会被当作重新训练后的模型目录；新的模型清单也不会替换冻结工件。

统一入口对输出路径执行排他性约束。锁定构建器通过 `os.O_EXCL` 创建结果和审计 JSON，写出前先拒绝已存在的目标；临时 ZIP 在同一输出目录生成，完成 CRC 与成员检查后再复制为最终文件。原始训练协调器同样要求模型根目录不存在，新推理要求输出目录不存在，`raw` 还要求模型目录和结果目录互不包含。这些限制看似严格，实质是为防止用户无意把上一次运行留下的模型、审计或提交文件混入本次结果。失败时应保留旧目录供检查，而不是覆盖它并损失证据。

锁定构建对哈希的检查顺序具有明确语义。首先验证传入的官方数据档案；其次验证冻结基座 ZIP 和 BPR32 检查点；随后验证 Dataset1 规则配置和两个官方测试 CSV；之后验证 Dataset1 经规则处理后的字节、Dataset2 精确组参考概率的流式字节、最终 Dataset2 字节以及最终 ZIP 的字节。每一层都比上一层更靠近输出。若早期输入哈希不符，程序不会继续做模型计算；若数值计算完成但最终封装哈希不符，程序也不会把该文件标记为成功。因而 `PASS` 的含义是所有证据链均一致，而不是仅表示脚本没有抛出异常。

检查点检查同样不仅是文件是否能被 NumPy 打开。BPR32 工件读取时要求固定 key 集合存在，检查 `kind` 和 `lineage` 文本、历史端点、32 维 factors、用户/目标 ID 的一维有序性、用户和目标 embedding 的二维 shape、目标偏置 shape、float32 数据类型、有限性以及显式保存参数与 state dict 参数的一致性。这个检查将“模型文件名称相同但内部数组被替换”的情况转化为明确错误。只有全部条件满足，构建器才把用户向量传给 Jittor 做归一化和余弦。

提交包自身的静态审计由 `audit_exact_package.py` 执行。它验证必需文件集合、禁止 `result.zip` 和 `data_A.zip` 被打包、禁止字节码缓存、禁止 PyTorch、TensorFlow、Paddle、MindSpore、MXNet、Keras、JAX、Flax 和 sklearn 的源码 import，并确认至少有 Jittor import。它还检查 `requirements` 中的版本字符串、PDF 文件签名和页数、报告 HTML 的中文正文长度、不含图像/SVG/旧式页眉，以及 `metadata` 中的赛道、队名、分数、名次和联系方式。静态审计不能替代运行时数值检查，但它能在 GPU 训练之前尽早发现包结构、框架合规或文档交付上的明显错误。

构建与训练收据选择 JSON 而非自由文本日志。JSON 包含固定键，如数据 SHA-256、源码/requirements 哈希、模型清单、运行时 Jittor 版本、CUDA 是否启用、输出路径和输出哈希，便于脚本读取、版本比对和人工复核。训练日志中的 loss 是优化过程的诊断，不会被用作“线上提升”的证据；新

推理生成的结果也不会被误写为 A 榜记录结果。只有锁定路径最终命中预先固定的结果哈希，才会在 `build_audit.json` 中给出 `decision=PASS`。

3.13 计算资源、内存控制与 Jittor 运行时行为

时序图候选排序既有大规模离散索引处理，也有密集的候选组神经网络计算。代码将两者分层处理以控制显存和内存。原始 CSV 的读取、稳定排序、ID 映射、候选去重、group 键构造和哈希由 NumPy/Pandas/Numba 在 CPU 侧完成；VAE 稀疏行、BPR embedding batch、图排序组特征和集合排序张量则在需要计算时以 batch 形式转换为 Jittor Var。Dataset1 训练 batch 为 512，VAE batch 为 128，BPR batch 为 32768，候选集合成员默认 batch 为 512；这些数值不是单纯为了追求吞吐，也限制了单次分配的张量大小。

社区余弦是锁定重建中唯一需要对大量测试候选碰撞对执行深度学习张量运算的部分。实现先用 NumPy 通过排序和分组找到确实需要比较的跨来源对，再将索引得到的 32 维用户向量按 131071 对一个批次送入 Jittor。这样不会构造一个“所有 153420 行两两比较”的二次规模张量，Jittor 只计算由同一时间和同一候选键筛选出来的有效对。输入向量在每批使用相同的 $1e-12$ 保护归一化，输出预先分配为 float32 数组；随后立即进行有限性验证。该实现既满足 Jittor 承担神经向量运算的要求，也将候选结构索引的内存成本保持在可审计范围。

运行脚本通过 `ML_CACHE_ROOT`、`JITTOR_HOME` 和 `HOME` 将 Jittor 编译缓存与运行缓存定向到调用方选择的数据盘路径。`JT_USE_CUDA=1` 会在子进程导入 Jittor 前设置，原始训练协调器还会在必要时发现系统 `nvcc` 并传递 `nvcc_path`。环境预检检查 CUDA 可用性并对一个固定 float32 向量执行平方和探针；这比只检测包是否安装更接近实际编译与执行条件。若 CUDA、cuDNN、版本或探针失败，程序停止而不以 CPU 或其他框架静默降级。

Jittor 的元算子编译缓存可能取决于驱动、编译器和本地目录，因此缓存本身不属于提交物，也不被作为验收工件。提交包锁定的是源码、Jittor 版本、依赖版本、模型数组、输入档案、规则和输出字节；缓存只是一种可再生的运行时加速产物。这个区分使包保持在规规定体积内，也避免把机器相关的二进制中间文件误当作模型或数据的一部分。运行完成后，审核者可以保留自己的缓存用于重复验证，而不影响清单对提交源文件的完整性判断。

3.14 审核复核顺序与结果解释

建议审核者按“解压结构—静态审计—环境预检—锁定重建—结果哈希—原始训练源码”的顺序复核。前两步首先确认包内不存在官方数据、测试标签和最终提交文件，且深度学习 import 只有 Jittor；环境预检随后确认 Python、Jittor、CUDA、cuDNN 和数值探针满足固定条件；`verify` 则从官方数据与包内冻结工件生成一个新的输出目录。该顺序可以将文件损坏、依赖不匹配、数据替换、规则变更和数值输出差异定位到不同层级。若最终 ZIP 哈希匹配，结论是记录结果被精确重建；若原始训练生成不同模型哈希，结论是新训练运行已经完成但不应被错误表述为历史记录的字节复刻。

报告内所有“固定”“锁定”“精确”的表述仅针对上述包含官方 `data_A.zip`、包内 `base_result.zip`、BPR32 检查点、规则文件和确定性封装的重建链路。它不表示不同 GPU 驱动、不同 Jittor 版本或从头训练的随机过程必然产生相同参数。通过明确限定这个结论范围，提交材料既能提供可运行的原始 Jittor 训练代码，也能避免将未经证明的训练再现性扩大为对历史工件的无条件承诺。

3.15 文件到算法逻辑的可追踪映射

为了避免“目录中有很多脚本但无法判断哪个脚本参与结果”的问题，本包将核心路径映射为清晰的文件链。`code/main.py` 只负责解析子命令并转发；`run_verify.sh` 是记录结果路径的外层门控；`run_inference.sh` 负责 CUDA 运行时准备、精确环境检测和构建器调用；真正生成结果的文件是 `build_community_residual_submission.py`。该构建器显式导入 `Dataset1`、`Dataset2` 两个后处理模块，读取模型和基座，完成 Jittor 社区余弦、候选内融合、规范序列化和固定 ZIP 写出。因而从入口到最终输出不存在依赖 shell 历史、隐式 notebook 或人工复制 CSV 的未记录步骤。

`Dataset1` 的文件职责也被拆分。`raw_training/dataset1/run.py` 包含时序图统计、特征构造、Jittor `Net/NetAttn` 与候选组训练的实现；`train_d1_cf_jittor.py` 负责将训练参数、数据路径和输出模型文件传入该实现；`score_d1_ensemble_jittor.py` 读取两个 seed 的模型并写出集成分数；`dataset1/d1_source_support_postprocess.py` 在冻结基座上执行来源支持规则。训练、打分和锁定规则分别位于不同文件，避免一段后处理逻辑被误解为训练网络的一部分，也使审计者能单独验证候选表只在后处理阶段作为无标签结构输入。

`Dataset2` 的模型路径具有相同的分工。`train_vae_jittor.py` 定义 MultVAE/RecVAE、加载稀疏时间衰减矩阵并导出 Jittor 参数；`train_bpr_jittor.py` 定义 BM25 权重、BPR loss 和 embedding 导出；`score_model_jittor.py` 为保存的表示计算候选分数。`d2_pool_ranker_production_jittor.py`、`d2_v2_set_production_jittor.py`、`d2_multislice_set_raw_jittor.py`、`d2_multislice_transformer_raw_jittor.py` 与 `d2_warm_residual_raw_jittor.py` 依次覆盖候选池、集合、多切片注意力和热节点残差成员。`build_result.py`、`build_set_v2_result.py` 和最终构建脚本负责融合这些成员，而 `dataset2/d2_exact_group_postprocess.py` 则仅实施冻结后处理。

工具目录的职责是保护运行条件而不是参与模型打分。`check_exact_environment.py` 检查版本与 Jittor CUDA 探针；`prepare_cuda_runtime.sh` 准备调用方可写的 CUDA/cuDNN 运行时路径；`audit_exact_package.py` 审核文件树、依赖、框架 import、报告和包边界；`build_exact_package.py` 仅在所有条件通过后压缩提交根目录。原始训练路径还保留 `check_raw_pipeline_contract.py`、`check_environment.py`、`verify_raw_training.py` 和 `verify_fresh_raw_run.py`，用于检查新运行的输入/输出约束。这种工具分层保证“审计代码能否修改模型输出”和“模型代码能否绕过审计”是两个可独立检查的问题。

提交根目录的 README、code/README 和本报告承担不同粒度的说明责任。根 README 给出最短安装和复现入口，code/README 给出子命令、目录和输出说明，本报告则解释模型选择、算法细节、输入边界、版本与审计语义。三者都指向同一 `code/main.py`，不会要求审核者根据文档自行拼接未列出的参数。根目录的 `submission_metadata.json` 保存赛道、团队、名次、分数与联系信息，供审计器逐字段验证；它不包含数据、模型权重或结果概率。

3.16 官方数据解析、输出格式与确定性序列化

官方数据以 ZIP 传入，代码不会预先把完整数据包复制进提交压缩包。`Dataset1` 与 `Dataset2` 的候选测试表均要求 header 为 `src,time,c1,...,c100`，后处理使用二进制流读取第一行 header，再逐行以 ASCII 逗号格式解析为 int64 数组。`Dataset1` 固定接受 61,051 行、`Dataset2` 固定接受 153,420 行，每行必须正好具有 102 列。行数、列数、候选 ID 范围、行内候选唯一性或官方测试成员哈希不符合预期时会

立即失败。此解析方式没有依赖 CSV 软件的默认类型推断，能够避免浮点转换、缺失值填补或列重排造成的隐式差异。

在训练阶段，Pandas 只用于从 ZIP 成员读取官方训练表的指定列并构造时间过滤后的 NumPy 数组。模型计算前，ID 映射均用 `np.searchsorted` 对经过排序的用户/目标数组完成；再通过有效位掩码确认每个被映射的 ID 真实存在。该方式不会因 Python 字典的插入顺序或未知 ID 的默认值产生不稳定索引。候选集合后处理中的稳定排序显式指定 `kind=stable`，在候选重复时保持原列次序，之后再以 `np.put_along_axis` 恢复原始 100 列。即便官方候选表恰有重复项，代码的传播逻辑也有确定行为；Dataset1 还更严格地要求每行候选无重复。

提交输出的两个 CSV 不带表头，每一行恰好写出 100 个十进制概率，以逗号分隔并以换行结束。概率写出前经过有限性、非负性和候选内归一化检查；锁定路径使用 `f"{value:.8f}"` 固定为八位小数。这一约束的取舍是放弃任何依赖本地 locale、NumPy printoptions 或 Pandas 浮点格式的自由度，换取跨运行结果能被逐字节验证。写入 ZIP 时，成员名不带目录前缀，成员顺序固定为 Dataset1 后 Dataset2，压缩方法、时间戳、权限、extra 字段和 comment 都显式设置。代码随后重新打开临时 ZIP，检查 CRC、成员列表和两个成员哈希，然后才使用排他方式写入目标路径。

输出的 `build_audit.json` 与 `result.zip` 分开保存，便于审核者在不展开大型 CSV 的情况下确认必要事实：官方档案哈希、冻结基座哈希、模型哈希、两个官方测试成员哈希、规则哈希、Jittor 与 NumPy 版本、CUDA 是否启用、社区余弦阈值、被排除的同来源/未知用户对计数、Dataset1 改变行数、Dataset2 top1 变化行数以及最终结果哈希。诊断字段不是评分依据，也不含隐藏标签；它只是将原本分散在执行过程中的可验证状态集中到一份机器可读收据中。

因此，复现者可以从不同层次验证输出。最轻量的检查是打开 ZIP 并确认恰有两个 CSV；格式检查进一步确认矩阵形状和每行概率和；构建审计确认输入和运行时都满足锁定条件；最后通过 SHA-256 将结果与记录对象比较。四个层次由弱到强，各自回答“可解压吗”“符合提交格式吗”“生成过程是否在规定的边界内”“是否为记录结果的精确字节副本”四个不同问题。把它们写成独立检查而非一个模糊的成功标志，是本包可人工审核性的核心设计之一。

4. 环境配置与依赖说明

4.1 目标平台与版本固定

目标平台为 Ubuntu 22.04、Python 3.10、NVIDIA RTX 4090 或等价 CUDA GPU，以及 CUDA 12.4 兼容的驱动和工具链。锁定重建不是“尽量兼容任意版本”的脚本：`check_exact_environment.py` 在写出结果之前要求 Python 主次版本为 3.10、Jittor 为 1.3.10.0、NumPy 为 1.26.4、Pandas 为 2.2.3、Numba 为 0.66.0、nvidia-cudnn-cu12 为 8.9.7.29，并要求 `nvcc` 和 Jittor CUDA 都可用。随后它执行固定向量的 Jittor 平方和探针，只有结果为 14.0 才继续。

表 3 requirements.txt 中声明的依赖

依赖	固定或兼容范围	用途
jittor	==1.3.10.0	全部深度学习训练、Jittor 参数读取和社区余弦计算
numpy	==1.26.4	数组、概率、哈希前数值检查和检查点载入
pandas	==2.2.3	原始训练阶段的官方 CSV 读取与时间字段处理
numba	==0.66.0	Dataset1 特征处理中的 CPU JIT 内核
nvidia-cudnn-cu12	==8.9.7.29	系统 CUDA 缺少开发头文件时的调用方本地包装层
jittor-geometric	>=0.1.0	竞赛兼容环境声明；当前源码不导入
scikit-learn	==1.5.2	竞赛兼容环境声明；当前源码不导入

4.2 安装步骤与路径约束

```
cd contest1_sader_007
python3.10 -m venv /data1/contest1-repro-env
source /data1/contest1-repro-env/bin/activate
python -m pip install -r requirements.txt
export JITTOR_HOME=/data1/contest1-jittor-cache
export ML_CACHE_ROOT=/data1/contest1-runtime-cache
export CUDA_VISIBLE_DEVICES=0
```

上述命令把虚拟环境、Jittor 编译缓存和运行时缓存放在数据盘。运行脚本会以调用方设置的 `CUDA_VISIBLE_DEVICES` 为准，默认值为 0；多人共享服务器时应先选择空闲 GPU。`prepare_cuda_runtime.sh` 优先使用系统 CUDA；若系统 SDK 未暴露 `cudnn.h` 或 `libcudnn.so`，脚本从已安装的 cuDNN Python 包创建位于 `ML_CACHE_ROOT/cuda_toolchain` 的 `include`、`lib64` 和 `nvcc` 包装层，不改写系统目录。

4.3 JittorGeometric、scikit-learn 与框架合规

依赖清单列出 JittorGeometric 和 scikit-learn，以覆盖竞赛要求中提出的图学习和常用数值环境。当前提交包的训练、推理和锁定重建均不导入这两个包：Dataset1 图排序直接由 Jittor 的 `embedding`、`linear` 层和 Jittor 张量运算实现；Dataset2 模型同样全部使用 Jittor。静态审计会扫描包内 Python `import`，拒绝 PyTorch、TensorFlow、Paddle、MindSpore、MXNet、Keras、JAX 和 Flax。Pandas、Numba、NumPy 仅承担数据和数值处理，不构成深度学习框架。

4.4 环境失败的处理原则

若解释器版本、依赖版本、GPU 可见性、nvcc、cuDNN 或 Jittor CUDA 探针不匹配，脚本应在模型计算和结果写出前停止。若官方 `data_A.zip` 的 SHA-256 与代码内常量不一致，静态审计和构建器均会拒绝。若 CUDA 浮点行为导致候选概率序列化发生差异，最终 ZIP 哈希不会匹配，构建器同样会失败。该“失败关闭”策略保证输出成功时可证明一致，输出失败时不会混入未经验证的近似结果。

5. 运行步骤

5.1 记录 A 榜结果的精确重建

```
cd contest1_sader_007
python code/main.py verify \
  --data /data1/songwentao/DL/data/data_A.zip \
  --output /data1/contest1-verify
```

`--data` 指向未修改的官方数据档案；`--output` 指向必须不存在的新目录。该命令是审计和重建的推荐入口。它依次执行清单验证、静态包审计、官方数据哈希检查、CUDA/Jittor 预检和锁定构建，并写出 `/data1/contest1-verify/result.zip` 和 `build_audit.json`。构建成功意味着最终 ZIP 的 SHA-256 等于 `d36facee996b5d45806dd6e1d80f8a48883e505f57c8d8d842b9626a50e8e7ce`。

如仅需在已完成静态审计后重复锁定推理，可运行下列命令。它仍然验证环境、官方数据、冻结基座、BPR32 检查点、规则文件和最终哈希；区别只在于不再事先运行完整包审计。

```
python code/main.py infer \
  --data /data1/songwentao/DL/data/data_A.zip \
  --output /data1/contest1-output
```

5.2 Dataset1 原始训练

```
python code/main.py train \
  --data /path/to/data_A.zip \
  --models /data1/contest1-d1-models \
  --dataset dataset1 --cuda
```

此命令进入 `run_train.sh`，先执行 Jittor CUDA 环境预检和原始管线契约检查，再调用 `raw_training/main.py train`。`--dataset dataset1` 仅训练两个图排序成员；`--models` 必须是不存在的目录，训练输出写入 `base_models/dataset1/`，包括两个固定种子的模型和 `training_manifest.json`。固定的 `groups`、`valid`、`epochs` 和 `batch` 参数见第 2.4 节，命令行没有覆盖这些记录协议参数的入口，以避免无意改变训练配置。

5.3 Dataset2 原始训练

```
python code/main.py train \
  --data /path/to/data_A.zip \
  --models /data1/contest1-d2-models \
  --dataset dataset2 --cuda
```

Dataset2 训练顺序由协调器固定：生产 VAE/BPR 成员、历史切分成员、`pool`、`set`、多切片 `set`、多切片 Transformer、`warm residual`、`legacy` 组件和社区 BPR32。`--cuda` 是必选约束，缺少该参数时协调器直接报错；它防止某些子模块在 CPU 环境中以不同性能或数值路径运行。模型目录记录每个文件的 SHA-256，因此重新训练后可以检查工件是否被后续操作替换。

5.4 全量训练与新模型推理

```
python code/main.py train \
  --data /path/to/data_A.zip \
  --models /data1/contest1-models \
  --dataset all --cuda

python code/main.py fresh-infer \
  --data /path/to/data_A.zip \
  --models /data1/contest1-models \
  --output /data1/contest1-fresh-output
```

第一条命令训练两个数据集的全部保留成员；第二条命令以该模型目录生成新的候选排序结果。新推理过程会先依次计算 Dataset1 集成分数、VAE/BPR 分数、pool/set/multislice/Transformer 元分数，再逐步调用基座融合器与最终后处理。输出目录中的 `inference_manifest.json` 绑定数据哈希、模型清单、源码清单、requirements 哈希和结果文件哈希。新训练路径的结果用于独立复核，不被宣称为历史 A 榜字节等价。

5.5 训练、推理与收据的一体化执行

```
python code/main.py raw \
  --data /path/to/data_A.zip \
  --models /data1/contest1-raw-models \
  --output /data1/contest1-raw-output
```

`raw` 调用 `run_raw.sh`，按“训练—模型验证—新推理—新推理验证—提交结构验证”的顺序执行。模型目录与输出目录必须互不包含、互不相同且在调用前不存在。脚本最终写入 `raw_training_verification.json` 和 `fresh_run_verification.json`；二者明确标记新训练不等同历史精确快照，从而避免把新的实验运行与锁定 A 榜重建混淆。

5.6 输出文件与提交格式检查

锁定构建和新推理均把最终提交写为 ZIP。有效 ZIP 的成员名严格为 `dataset1.csv`、`dataset2.csv`，不能包含路径前缀或额外临时文件。验证器加载两个 CSV 后检查形状、有限性、非负性和每行概率和；随后检查 ZIP CRC。锁定路径在此基础上追加完整 ZIP 的 SHA-256 检查。该顺序确保“格式正确”和“记录结果字节一致”是两个独立的结论：前者适用于新训练推理，后者只适用于 `verify` 锁定重建。

表 4 运行接口、输入、输出与验证目标

命令	主要输入	主要输出	验证语义
verify	官方数据；包内冻结工件	result.zip; build_audit.json	记录结果精确重建
infer	官方数据；包内冻结工件	result.zip; build_audit.json	直接锁定推理
train	官方数据；新模型目录	Jittor 检查点; training_manifest	新模型训练
fresh-infer	官方数据；新模型目录	result.zip; inference_manifest	新模型提交格式验证
raw	官方数据；两个新目录	模型、结果和双重收据	端到端新运行审计

6. 其他补充

6.1 关键超参数的含义与取值

表 5 关键超参数及其训练含义

组件	参数	含义
Dataset1 图排序	seeds=20260705/20260715; groups=80000; valid=20000; epochs=16; batch=512	双成员随机种子、训练/验证查询规模、迭代轮数和候选批量
MultVAE	epochs=10; batch=128; hidden=600; latent=200; dropout=0.5	隐变量稀疏交互建模容量与正则化
RecVAE	cycles=5; batch=128; hidden/latent=600/200; 另有 800/400 配置	循环优化轮数和模型容量
BM25-BPR	factors=256; epochs=20; batch=32768; negatives=8; lr=0.002; reg=0.0001	排序嵌入维度、负采样数、优化步规模和正则强度
社区 BPR32	factors=32; epochs=3; batch=32768; negatives=4; lr=0.002	锁定社区余弦残差所需用户表示

这些超参数的作用可从两个层次理解。模型容量参数（hidden、latent、factors）决定编码或嵌入的表达能力；优化参数（epochs/cycles、batch、negatives、learning rate、regularization）决定在固定数据和随机种子下的学习轨迹。记录结果的锁定重建不会在运行时重调上述参数；后处理的 0.05 和 0.02 系数同样被固定，并且没有由测试标签驱动的参数搜索分支。

6.2 可复现性机制

提交包提供三类相互补充的证据。第一类是文件完整性证据：MANIFEST.sha256 覆盖源代码、规则、模型、PDF 和依赖文件，外层 ZIP 还接受 CRC 检查。第二类是输入边界证据：官方数据档案、冻结基座、BPR32 检查点和规则文件都具有固定哈希；测试标签没有被打包或读取。第三类是输出证据：构建器检查每个中间概率矩阵的内容哈希和最终结果 ZIP 的哈希。三类证据覆盖“代码是否变化”“输入是否被替换”和“输出是否字节一致”三个不同问题。

Jittor 的编译缓存可能因主机、Python 路径、GCC 或 CUDA 工具链不同而重建，因此缓存目录不被放入提交包，也不作为结果工件。真正被锁定的是 Jittor 版本、CUDA 可用性探针、检查点内容和最终结

果。run_inference.sh 设置 JT_USE_CUDA=1，构建器再显式设置 jt.flags.use_cuda=1；若目标环境不能提供 CUDA Jittor，则不能声明锁定重建成功。

6.3 已知边界与审查注意事项

历史实验包含自动化任务编排和参数搜索，少量边缘候选的原始超参数状态没有逐一固化为独立文件。因此，本包不把“从零重新训练所有成员”描述为历史权重的字节级再现。相反，包内保留了记录结果所需的中间分数工件、社区检查点、锁定规则和输出哈希，verify 路径以这些可审计输入重建记录提交。原始训练源码仍完整提供，审核者可从官方数据执行 train 或 raw，但应以其独立清单和输出收据评估新运行。

JittorGeometric 的声明只用于满足兼容环境的依赖清单；当前代码没有把 JittorGeometric、scikit-learn 或任何非 Jittor 深度学习框架作为隐藏模型成员。若审核环境已按 requirements.txt 配置，两个包可以存在于环境中而不参与本项目的计算图。审计器关注的是源代码实际 import、数据读取边界和生成结果，而不是未被调用的兼容依赖名称。

6.4 合规声明

本项目的训练、推理和社区余弦计算使用 Jittor；CSV 读取和特征处理使用 NumPy、Pandas 和 Numba。项目没有使用外部数据集、测试标签、第三方预测结果或非 Jittor 深度学习框架。官方测试候选和时间仅按照代码所示方式用于候选内排序、分组和结构残差。提交包不携带 result.zip，运行者必须从官方数据和包内代码/工件生成结果，这使审计者能够检查完整输出过程。

参考文献

- [1] Yi L, Lei R, Mo F, et al. Future Link Prediction Without Memory or Aggregation. arXiv:2505.19408, 2025. (仅参考标题区、摘要、连续正文与图表的学术排版层级；本文未采用其模型代码、图像或正文内容。)
- [2] Rossi E, Chamberlain B, Frasca F, et al. Temporal Graph Networks for Deep Learning on Dynamic Graphs. arXiv:2006.10637, 2020. (时序图学习背景参考。)
- [3] Jittor Team. Jittor: a novel deep learning framework with meta-operators and unified graph execution. 本项目固定使用 Jittor 1.3.10.0 作为深度学习实现框架。